Information Systems Security Project

Στέργιος Λούκας, AEM: 10359

SQL injection fixes

register.php: i could not achieve an sql injection but this is still a bad practice so i used prepared statements to make the INSERT of (username, password) secure.

```
// Insert a new user
//comment out this line and make prepared statement
//$sql_query = "INSERT INTO login_users (username,password) VALUES ('{$new_username}','{$new_password}');";
//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query = "INSERT INTO login_users (username,password) VALUES (?,?);";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, "ss", $new_username, $new_password);
$result = mysqli_stmt_execute($statement);
mysqli_stmt_close($statement);
```

login.php: I used the recommended payload `xxx' OR 1=1; --` with a non-empty password and managed to extract all website/username/password (websites TABLE). Just as before, fixed it by using a prepared statement.

```
// xxx' OR 1=1; -- '
//$sql_query = "SELECT * FROM login_users WHERE username='{$username}' AND password='{$password}';";
//echo $sql_query;
// Check if the credentials are valid
//$result = $conn->query($sql_query);

$sql_query = "SELECT * FROM login_users WHERE username= ? AND password= ? ;";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, "ss", $username, $password);
mysqli_stmt_execute($statement);
$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
```

Dashboard.php: Same problem as previously (SQLi) solved in the same way. `mysqli_stmt_get_result()` is not needed for INSERT and DELETE operations but it does not cause breaking changes (commented out).

```
// Insert new web site
//$sql_query = "INSERT INTO websites (login_user_id,web_url,web_username,web_password) VALUES " .
            "((SELECT id FROM login_users WHERE username='{$username}'),'{$new_website}','{$new_username}','{$new_password}');";
//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query =  "INSERT INTO websites (login_user_id,web_url,web_username,web_password) VALUES " .
            "((SELECT id FROM login_users WHERE username=?),?,?,?);";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 'ssss', $username, $new_website, $new_username, $new_password);
mysqli_stmt_execute($statement);
//$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
$conn -> close();
```

```php
// Delete selected web site
//$sql_query = "DELETE FROM websites WHERE webid='{$webid}';";
//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query = "DELETE FROM websites WHERE webid=?;";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 's', $webid);
mysqli_stmt_execute($statement);
//$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
$conn -> close();
```

In this case `$result` is needed because there is a SELECT operation, so we use `mysqli_stmt_get_result()`.

```php
// Display list of user's web sites
//$sql_query = "SELECT * FROM websites INNER JOIN login_users ON websites.login_user_id=login_users.id WHERE login_users.username='{$username}';";
//echo $sql_query;
//$result = $conn->query($sql_query);
//echo htmlspecialchars($username);
$sql_query = "SELECT * FROM websites INNER JOIN login_users ON websites.login_user_id=login_users.id WHERE login_users.username=?;";
$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 's', $username);
mysqli_stmt_execute($statement);
$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
```

Notes.php: Same problem, same fix (SQL injection → prepared statements).

```php
//$sql_query = "INSERT INTO notes (login_user_id, note) ".
//           "VALUES ((SELECT id FROM login_users WHERE username='{$username}'), '{$new_note}')";

//echo $sql_query;
//$result = $conn->query($sql_query);

$sql_query = "INSERT INTO notes (login_user_id, note) ".
             "VALUES ((SELECT id FROM login_users WHERE username=?), ?)";

$statement = mysqli_prepare($conn, $sql_query);
mysqli_stmt_bind_param($statement, 'ss', $username, $new_note);
mysqli_stmt_execute($statement);
//$result = mysqli_stmt_get_result($statement);
mysqli_stmt_close($statement);
$conn -> close();
```

Here there is no need for a fix because of no direct user input (left it intact).

```php
// Display list of all notes/comments
$sql_query = "SELECT notes.note, login_users.username FROM notes INNER JOIN login_users ON notes.login_user_id=login_users.id;";
//echo $sql_query;
$result = $conn->query($sql_query);

echo "<h3>List of notes/comments</h3>";
```

References: https://www.youtube.com/watch?v=a7iQxCgqpoI

Plain text save fixes

register.php: here, passwords are saved as plain text, making them vulnerable to anyone that has access to the database. I will use the function `getPasswordHash_Hex($username, $password)` from `test_hash.php` to salt (salt == hashed username) and hash (with sha256) and then store passwords as salted hashes, securing them correctly.

First, I commented out the examples in `test_encrypt.php` and `test_hash.php` to include the files and use the functions: `hash_function` in `register.php`.

```php
//include('includes/hash_function.php');
function getPasswordHash_Hex($username, $password) {
    // Compute hash of salted-password (and salt) from username and password (in hex format)
    $salt = hash('sha256', $username);  // Compute salt as the hash of the username
    $saltedPwd = $salt . $password;     // Get a salted password by combining salt and password
    $hashedPwd = hash('sha256', $saltedPwd);    // Hash the salted password using SHA-256
    // Return the password hash and the salt
    return [
        'hash' => $hashedPwd,
        'salt' => $salt
    ];
}
```

```php
include('test_hash.php');
```

Then I used the function to store the hashed password.

```php
    // Get user submitted information
    $new_username = trim($_POST['new_username']);
    $new_password = trim($_POST['new_password']);
    $getHasedPwd = getPasswordHash_Hex($new_username, $new_password);
    $new_password = $getHasedPwd['hash']; //only save the hashed password and not the 'salt'
    mysqli_report(MYSQLI_REPORT_OFF);  // disable exceptions
```

After applying changes I successfully registered new users with hashed passwords.

u45	c629dee600a45132f5f430b9db399c29618e8665...
u50	2a990e084bfa9d43d8d16b52556a99cb13073eb...
sdadsadsa	11633539c2aa552f48144dbcbb0a21cd2254e7a...
stergiosloukas	63577649ad7aafccee6ea4aa0e0f826023ddc823...

After that I had to update `login.php` accordingly so that it follows the hashing logic to verify user input/login credentials. I included the function file here too and updated the logic.

```php
<?php
include('test_hash.php');
```

```php
else {
    // Get user submitted information
    $username = trim($_POST['username']);
    $password = trim($_POST['password']);
    $getHasedPwd = getPasswordHash_Hex($username, $password);
    $password = $getHasedPwd['hash'];
```

After applying changes I successfully registered and entered the system as a user with a hashed password stored.

Entries of stergiosloukas

No entries found.

website

Username

Password

Insert new website

Notes - announcements

Logout

Home page

31	stergiosloukas	63577649ad7aafccee6ea4aa0e0f826023ddc823...

Website Passwords: Another issue is that website passwords are not inserted as encrypted values, compromising them and making them easy to steal for someone with DB access.

login.php: First I derive the session key because while the username is available in the dashboard page, the password is only available in login.

```php
<?php
include('test_encrypt.php');
```

```php
//key derivation to encrypt/decrypt data in dashboard.php
$key = $_SESSION['encryption_key'] = deriveEncryptionKey($username, $password);
```

dashboard.php: Then I encrypt the web password using the session key before entering it in the DB.

```php
<?php
include('test_encrypt.php');
```

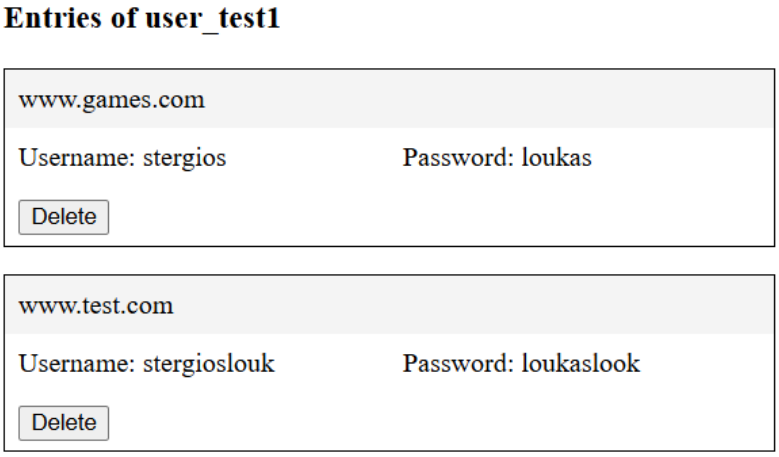
```php
//password encryption using the derived session key from login.php
$key = $_SESSION['encryption_key'];
$encrypted_password = encryptData($new_password, $key);
$new_password = $encrypted_password;
```

And when the password must be displayed it is decrypted using the reverse logic, with the same session key (because username and password remain the same).

```php
if (!empty($result) && $result->num_rows >= 1) {
    while ($row = $result -> fetch_assoc()) {
        //decrypt the password before displaying
        $key = $_SESSION['encryption_key'];
        $row["web_password"] = decryptData($row["web_password"], $key);
```

After applying the changes, I was able to store a encrypted web password and still view it as decrypted in the application.

23	34	www.games.com	stergios	+DQHV33t93BaIXgThmDLtWNIdG0Jmsyyp51iXC8VtIfigw==
24	34	www.test.com	stergioslouk	SkLIMR7hqbZqeAIhK5qz/vjnUU+S3z5xsk7INGcgIlg/8ly5nKI=



notes.php: notes could be encrypted/decrypted too but there is no need for it, as they are public for all logged in users.

References:

- https://www.youtube.com/watch?v=zt8Cocdy15c
- `passman/test_hash.php` → function `getPasswordHash_Hex($username, $password)`
- `passman/test_encrypt.php` → function `deriveEncryptionKey($username, $password)`
- `passman/test_encrypt.php` → function `encryptData($data, $key)`
- `passman/test_encrypt.php` → function `decryptData($data, $key)`

Cross Site Scripting (XSS) fixes

dashboard.php: displays unsanitized user-stored data making it vulnerable to stored XSS attacks. Still, only the logged in user can view that data so an attack is impossible, unless he self XSS, self harms by storing malicious content. Just in case I sanitized that output too, using `htmlspecialchars()`.

```
(!empty($result) && $result->num_rows >= 1) {
 while ($row = $result -> fetch_assoc()) {
    //decrypt the password before displaying
    $key = $_SESSION['encryption_key'];
    $row["web_password"] = decryptData($row["web_password"], $key);
    echo "<table border=0>";
    echo    "<tr style='background-color: #f4f4f4;'><td colspan=2>" . htmlspecialchars($row["web_url"]) . "</td></tr>" .
            "<tr><td>Username: " . htmlspecialchars($row["web_username"]) . "</td><td>Password: " . htmlspecialchars($row["web_password"]) . "</td></tr>";

    echo    "<tr><td><form method='POST' style='height: 3px'>" .
            "<input type='hidden' name='websiteid' value='" . htmlspecialchars($row["webid"]) . "'>" .
            "<button type='submit' name='delete_website'>Delete</button></form></td></tr>";

    echo    "<tr><td colspan=2 style=height: 20px;></td></tr>";
    echo "</table><p/>";
```

notes.php: On the other hand, `notes.php` stores all users' data and displays them unsanitized so it is imperative that they are sanitized first so that the malicious code/data does not end up running on another user's side. Therefore, I edited the display section to sanitize the output using `htmlspecialchars()` before displaying the notes of users.

```
if (!empty($result) && $result->num_rows >= 1) {
    while ($row = $result -> fetch_assoc()) {
        echo "<div class='note'>";
        //echo  "<div class='note-content'>" . $row["note"] . "</div>";
        //echo  "<div class='note-signature'> by " . $row["username"] . "</div>";
        //Sanitization against XSS
        echo "<div class='note-content'>" . htmlspecialchars($row["note"]) . "</div>";
        echo "<div class='note-signature'> by " . htmlspecialchars($row["username"]) . "</div>";
        echo "</div>";
    }
```

Testing and Confirmation: To confirm the fix, I did a full reset of the DB (deleted all data from `login_users`, `notes`, `websites`) and made two accounts, attacker, defender to test the XSS vulnerabilities.

pwd_mgr.login_users: 2 rows total (exact)

#	1 id	2 username	3 password
1	1	attacker	ae051a9330432e3ac12d91b2af0db8b542a00af7...
2	2	defender	2291461bc5f5c46259e8223827460efa0a55979f...

I then posted some malicious XSS payloads in notes as the attacker.

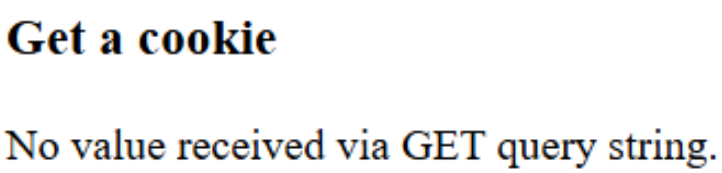
<h1 id="welcome"></h1>

by attacker

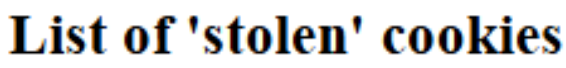
<script>alert("I HACKED YOU DUDE")</script>

by attacker

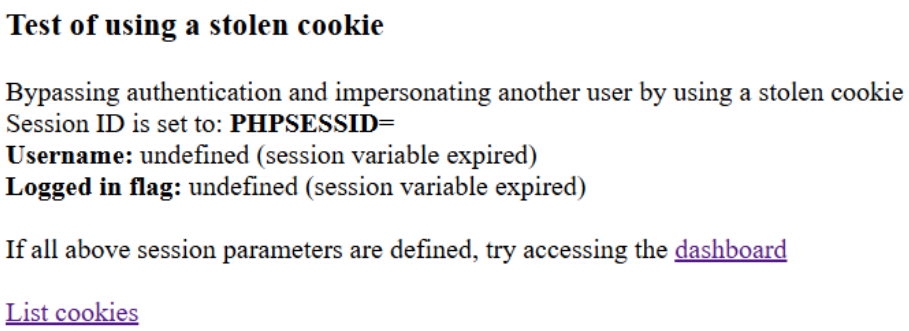
After entering as the defender and visiting the notes page, no alerts popped up in my browser. I also added a website/username/password entry to my account as the defender. I also checked the pages of the hacker's side after logging in and, as expected, no cookie was stolen.



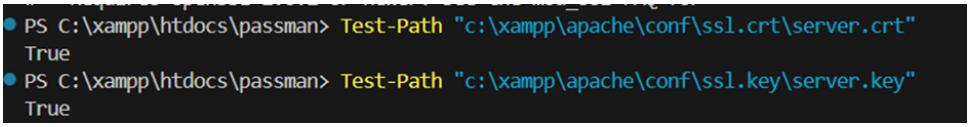
`Stolencookies.txt` also remains empty, just like list of stolen cookies.

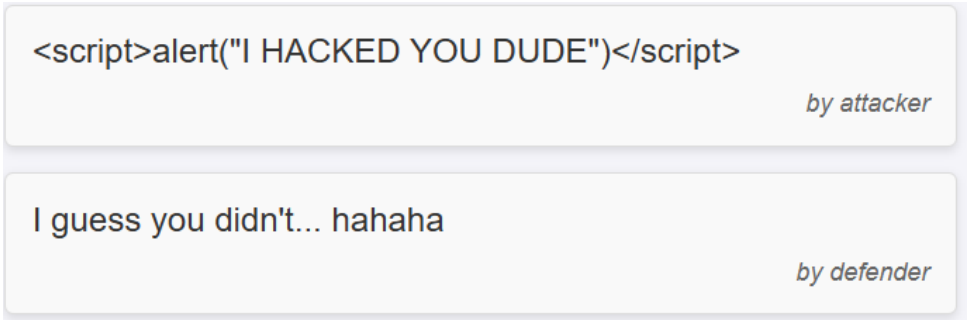


And `usecookie` page also lacks the cookie needed to hack into defender's page.



HTTP to HTTPS Migration: First I generated a self-signed SSL Certificate. Then, I enabled SSL in the Apache web server and configured it with the self-signed certificate (`server.crt` and `server.key`). Port 80 is listening to HTTP and Port 443 is listening to HTTPS after the new configuration.




<script>alert("I HACKED YOU DUDE")</script>
by attacker
I guess you didn't... hahaha
by defender

The end.